Name (Last, First): _____

These problems are about `40k_filesystem` and `41k_devices`. There are 2 problems on 4 pages; Problem 2 (41k) is **optional extra credit**. Answer each part in the space below it, and show your work.

1. **Filesystem (40k).** In 40k, a directory is an inode whose entries map names to inode numbers. Here is the code:

```
pub const NINODE: usize = 64;  pub const NDIRENT: usize = 16;   // inodes; entries per dir
pub const NAMELEN: usize = 14; pub const FILESIZE: usize = 128; // max name; max file size
pub const ROOT: usize = 1;                    // inode 1 is the root directory "/"
pub enum InodeKind { Free, File, Dir }
pub enum FsError { NotFound, AlreadyExists, NotADirectory, IsADirectory,
                   NoFreeInode, DirFull, NameTooLong, FileTooBig }
struct DirEnt { name: [u8; NAMELEN], len: usize, inum: usize, used: bool }
pub struct Inode { kind: InodeKind, size: usize, data: [u8; FILESIZE],
                   entries: [DirEnt; NDIRENT] }  // a Dir's name -> inode entries

fn alloc(&mut self, kind: InodeKind) -> Result<usize, FsError> {
    for i in ROOT..NINODE {                      // the first Free inode, from ROOT up
        if self.inodes[i].kind == InodeKind::Free {
            self.inodes[i] = Inode::new();       // a fresh, empty inode
            self.inodes[i].kind = kind;
            return Ok(i);
        }
    }
    Err(FsError::NoFreeInode)
}
pub fn dirlookup(&self, dir: usize, name: &[u8]) -> Result<usize, FsError> {
    if self.inodes[dir].kind != InodeKind::Dir { return Err(FsError::NotADirectory); }
    for e in &self.inodes[dir].entries {         // find the entry with this name
        if e.used && e.len == name.len() && &e.name[..e.len] == name {
            return Ok(e.inum);
        }
    }
    Err(FsError::NotFound)
}
pub fn dircreate(&mut self, dir: usize, name: &[u8], kind: InodeKind)
                -> Result<usize, FsError> {
    if name.len() > NAMELEN { return Err(FsError::NameTooLong); }
    match self.dirlookup(dir, name) {            // reject duplicates
        Ok(_) => return Err(FsError::AlreadyExists),
        Err(FsError::NotFound) => {}             // not there yet: go ahead
        Err(e) => return Err(e),                 // any other error: pass it up
    }
    let mut slot = None;                         // a free entry in dir
    for i in 0..NDIRENT {
        if !self.inodes[dir].entries[i].used { slot = Some(i); break; }
    }
    let slot = slot.ok_or(FsError::DirFull)?;    // None -> return Err(DirFull)
    let inum = self.alloc(kind)?;                // Err -> return that error now
    let mut nm = [0u8; NAMELEN];                 // the name, padded with zeros
    nm[..name.len()].copy_from_slice(name);
    self.inodes[dir].entries[slot] = DirEnt { name: nm, len: name.len(), inum, used: true };
    Ok(inum)
}
```

(a) Right after `fs.init()`, inode 1 is the (empty) root directory and every other inode is `Free`. Fill in what each call returns, in order. The first row is done for you.

	Call	Returns
1	`fs.dircreate(ROOT, b"hello", InodeKind::File)`	`Ok(2)`
2	`fs.dircreate(ROOT, b"sub", InodeKind::Dir)`	
3	`fs.dircreate(ROOT, b"hello", InodeKind::File)`	
4	`fs.dirlookup(ROOT, b"sub")`	
5	`fs.dircreate(2, b"x", InodeKind::File)`	
6	`fs.dirlookup(3, b"notes")`	

Example, row 1: `"hello"` isn't in the root yet, so `dirlookup` returns `Err(NotFound)` and `dircreate` goes ahead. `alloc` scans from `ROOT = 1`; inode 1 is the root (a `Dir`), so the first `Free` inode is 2: `Ok(2)`.

(b) A student drops the length check from `dirlookup`:

```
if e.used && &e.name[..name.len()] == name {     // was: e.used && e.len == name.len() && ...
```

With `"hello"` in the root (row 1), what does `fs.dirlookup(ROOT, b"hell")` return now, and why? What does `fs.dircreate(ROOT, b"hell", InodeKind::File)` then return?

2. **Devices (41k).** ***Optional, extra credit.*** 41k's UART driver talks to the serial chip through its registers, one byte each, at `UART0 + offset`:

```
const UART0: usize = 0x1000_0000;   // where the UART's registers start
const RBR: usize = 0;          // Receive Buffer Register: the byte received (read)
const THR: usize = 0;          // Transmit Holding Register: a byte to send (write)
const LSR: usize = 5;          // Line Status Register: the chip's status flags
const LSR_DR: u8 = 1 << 0;     // Data Ready: a received byte is waiting in RBR
const LSR_THRE: u8 = 1 << 5;   // THR Empty: the chip can take another byte

unsafe fn reg_read(off: usize) -> u8 { read_volatile((UART0 + off) as *const u8) }
unsafe fn reg_write(off: usize, val: u8) { write_volatile((UART0 + off) as *mut u8, val); }

pub fn tx_ready() -> bool { unsafe { reg_read(LSR) & LSR_THRE != 0 } }
pub fn rx_ready() -> bool { unsafe { reg_read(LSR) & LSR_DR != 0 } }
pub fn putc(c: u8) {
    while !tx_ready() {}               // wait until the chip can take a byte
    unsafe { reg_write(THR, c) }
}
pub fn getc() -> Option<u8> {
    if rx_ready() { Some(unsafe { reg_read(RBR) }) } else { None }   // nothing yet: None
}
```

(a) For each value read from `LSR`, what do these functions return? The first row is done for you. *Hint:* write the value in binary, then look at the two bits the driver tests:

`LSR` bit	7	6	5	4	3	2	1	0
meaning	–	–	`THRE`	–	–	–	–	`DR`

`THRE = 1 << 5 = 0x20` the chip can take another byte
`DR = 1 << 0 = 0x01` a received byte is waiting
– other status bits: ignore them

`LSR`	`tx_ready()`	`rx_ready()`	`getc()`
`0x60`	`true`	`false`	`None`
`0x61`			
`0x01`			

Example, row 1: `0x60 = 0110_0000` in binary. Bit 5 (`THRE`) is 1, so `tx_ready()` is `true`; bit 0 (`DR`) is 0, so `rx_ready()` is `false` and `getc()` returns `None`. (This is the UART right after start-up: ready to send, nothing received.)

(b) (i) `RBR` and `THR` are both offset 0, the same address. When the driver uses offset 0, what decides whether it reads the received byte or sends a byte? Point to the code. (ii) `putc` waits for `tx_ready()` before writing `THR`. What could go wrong on a real UART if it wrote right away?